Drift Telemetry System

Author: Radut Dragos-Andrei

1. Introduction

The Drift Telemetry System is an ESP32-based embedded device designed to capture, process, and transmit real-time vehicle dynamics data during drift sessions.
The system combines inertial measurement (IMU), GPS-based positioning and speed, local OLED feedback, and cloud persistence via Firebase Realtime Database, providing a complete picture of each drive.

2. System Architecture

Layer	Components	Responsibility
Embedded firmware	ESP32 + MPU6050 + NEO-6M + SSD1306	Sensor reading, signal processing, cloud publishing
Cloud backend	Firebase Realtime Database	Persistent storage, remote config, command relay
Dashboard	Web application (local)	Live telemetry display, run history, map view

3. Hardware

3.1 Components used

Component	Part / variant	Purpose
Microcontroller	ESP32 DevKit v1	Main processor, WiFi, UART, I2C host
IMU	MPU6050 (I2C, 0x68)	3-axis gyroscope + 3-axis accelerometer + die temperature
GPS receiver	u-blox NEO-6M (UART, 9600 baud)	Position, speed, course-over-ground
Display	SSD1306 128×32 OLED (I2C, 0x3C)	Local status — state, yaw rate, lateral G, WiFi/FB flags
Power	USB 5V → on-board 3.3V LDO	Powers all components

3.2 Wiring table

Component	Pin	ESP32 GPIO	Notes
MPU6050	SDA	GPIO 21	I2C data — shared bus
MPU6050	SCL	GPIO 22	I2C clock — shared bus
MPU6050	VCC	3V3	
MPU6050	GND	GND	
SSD1306	SDA	GPIO 21	Shared with MPU6050

SSD1306	SCL	GPIO 22	Shared with MPU6050
SSD1306	VCC	3V3	
SSD1306	GND	GND	
NEO-6M	TX	GPIO 16 (RX2)	GPS → ESP32 UART2 RX
NEO-6M	VCC	3V3	3V3
NEO-6M	GND	GND	

4. Functioning and Application Logic

The system operates as a continuous sensing and publishing pipeline. At boot, the ESP32 initializes the I2C bus, OLED display, and MPU6050, then connects to WiFi and synchronizes time over NTP before authenticating with Firebase. GPS is started on UART2 in parallel. The final boot step is IMU calibration: the device collects 200 samples over two seconds while stationary to compute the mean bias for all six IMU axes (accelerometer X/Y/Z and gyroscope X/Y/Z). These biases are subtracted in real-time from every subsequent reading, and a warning is raised if the residual yaw bias exceeds 1.5 °/s. Calibration can be re-triggered remotely at any time without rebooting.

Once running, the main loop executes without blocking delays. GPS bytes are consumed on every iteration to avoid UART buffer overflow. Every 100 ms (10 Hz), the IMU is read, derived quantities are computed, and the run state is evaluated. Cloud tasks run on independent timers: live telemetry uploads every 750 ms, run samples every second, and a status heartbeat every two seconds.

Signal processing happens entirely on-device. The raw gyroscope Z reading is bias-corrected and converted to yaw rate in degrees per second (`yawRateDps = (gyro.z − biasZ) × 57.2958`). The accelerometer Y axis is bias-corrected and normalised to lateral g-force (`latG = (accel.y − biasY) / 9.81`). Vehicle heading is maintained by a complementary filter that integrates the gyroscope at high rate and applies a slow GPS correction (`ψ += 0.05 × wrapTo180(χ_GPS − ψ)`) whenever speed exceeds 5 km/h — below that threshold GPS course is dominated by multipath noise and is ignored. The sideslip drift angle β is the angular difference between where the car points (heading ψ) and where it actually moves (GPS course χ), smoothed by a low-pass filter with α = 0.15 and decayed to zero when GPS is unavailable. A boolean **drifting** flag is raised when both

`|yawRate| ≥ 8 °/s` and `|latG| ≥ 0.20 g` are true simultaneously; the dual threshold prevents false positives from parking manoeuvres (high yaw, low G) and road camber (low yaw, non-zero G).

Run detection is speed-based: a run starts when GPS speed stays above 5 km/h for 700 ms and ends when speed drops below that threshold for more than 5 seconds. This records every drive, not just drift episodes, so the dashboard can render colour-coded route lines where the `drifting` flag changes. At run start, an initial document is written to Firebase with the GPS coordinates and zeroed summary fields. During the run, each sample uploads position, speed, IMU values, heading, drift angle, and the drifting flag. The summary sub-document is maintained with running accumulators (true maximums and running averages for speed, drift angle, yaw rate, and lateral G) rather than overwriting with the current sample, so it always reflects the full run history.

All data is stored under `/devices/drift-01/` in Firebase Realtime Database. The `/live` node provides a real-time snapshot for the dashboard display. The `/status` node publishes device health (state, WiFi RSSI, calibration flag, last seen timestamp. Each run is stored under `/runs/{id}` with per-sample data nested under `samples/` and aggregate statistics under `summary/`. The firmware also polls `/commands` (supporting `calibrate` and `reset` actions) and `/config`, allowing all detection thresholds to be adjusted remotely without reflashing.

5. Dashboard

The dashboard is a browser-based React application that connects to the same Firebase Realtime Database used by the firmware. It provides live monitoring, run history, and remote control — but no business logic: drift detection, run lifecycle, and all sensor processing remain entirely on the ESP32. The browser and the device communicate exclusively through shared database paths.

Main features

- **Live telemetry panel** — speed, yaw rate, lateral G, drift angle, and temperature updated at ~1.3 Hz from the `/live` node.
- **Live IMU charts** — scrolling Chart.js line charts for yaw rate and lateral G, maintaining a ring buffer of up to 120 points.
- **Device status banner** — derived device state (BOOT / CALIBRATING / READY / RUNNING / ERROR), stale-connection detection (triggered when `lastSeen` is more than 15 s old), and WiFi RSSI quality indicator.
- **Remote commands** — Calibrate and Reset buttons write to `/commands` with a monotonically incrementing `requestId`. The dashboard tracks whether the firmware has acknowledged the command by comparing `requestId` against `status.lastAckRequestId`.

- **Run history** — expandable table of all runs stored under `/runs`, each with a detail drawer containing per-run charts, a summary tile grid, a segment timeline, a GPS route map, CSV export, and freeform notes.
- **Route map** — Leaflet map overlaid on OpenStreetMap tiles. The track is coloured by drift state (blue = normal, orange = drifting). When GPS fix drops mid-run, the map dead-reckons position from heading and speed to fill the gap seamlessly.
- **Threshold configuration** — a config panel reads from and writes directly to `/config`, allowing all detection thresholds to be changed without reflashing.

6. Libraries and Dependencies

Library	Version	Used for
mobizt/FirebaseClient	latest	Firebase RTDB auth + set/get; UserAuth, RealtimeDatabase, AsyncClientClass
bblanchon/ArduinoJson	^7.0.0	Build JSON payloads for atomic multi-field writes
mikalhart/TinyGPSPlus	latest	NMEA sentence parsing from NEO-6M UART stream
adafruit/Adafruit MPU6050	latest	MPU6050 I2C driver (read, configure range/filter)
adafruit/Adafruit Unified Sensor	latest	Sensor abstraction layer (dependency of MPU6050)
adafruit/Adafruit SSD1306	latest	OLED I2C driver
adafruit/Adafruit GFX Library	latest	Graphics primitives (dependency of SSD1306)

7. Known Limitations

- **GPS cold start latency:** Without VBAT backup, every reboot incurs a 30–90 s acquisition delay. Runs that start before a GPS fix will have lat/lon = 0 and unreliable heading/driftAngle until the fix is established.
- **Drift angle accuracy at low speed:** GPS course is suppressed below 5 km/h, so drift angle decays to zero while parked. This prevents noise but also means the angle is unavailable during slow manoeuvres.
- **Heading divergence on sharp initial turns:** If the GPS heading initializes during a turn rather than straight-line motion, the complementary filter will take several seconds to converge, causing transiently large drift angle readings.
- **Run closure on power loss:** If the device loses power mid-run, finishRun() is never called and endedAt / durationMs remain 0. The run data is intact; only the closing metadata is missing.

8. References

1. https://ocw.cs.pub.ro/courses/iothings/laboratoare/2025/lab1
2. https://ocw.cs.pub.ro/courses/iothings/laboratoare/2025/lab9
3. bblanchon/ArduinoJson — https://arduinojson.org/
4. mikalhart/TinyGPS++ — https://github.com/mikalhart/TinyGPSPlus
5. https://randomnerdtutorials.com/arduino-mpu-6050-accelerometer-gyroscope/